

Section 7: Practical Exercise - Hello CanMV

1. Course Objectives

In this lesson, we will create an application called "Hello World" from scratch and run it on the K230.

2. Preparation

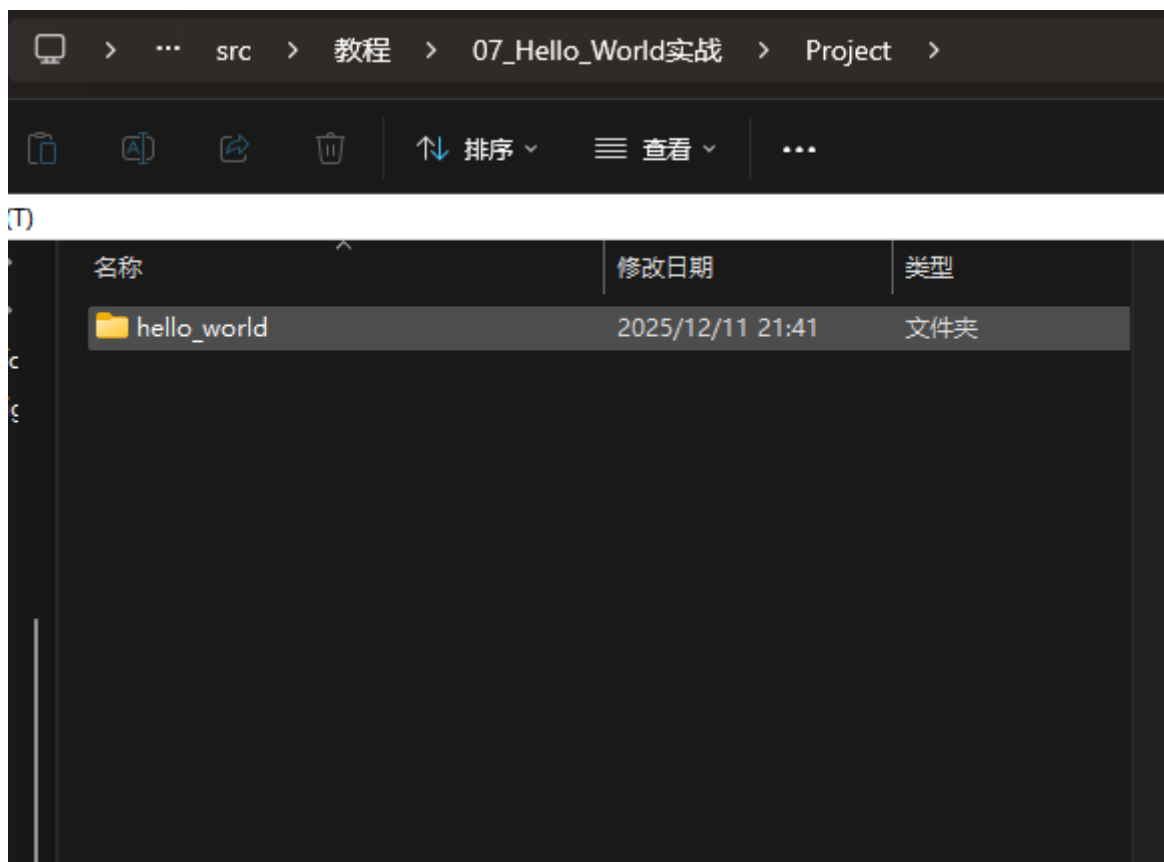
Please ensure you have connected to the development board and can access the `/sdcard` directory.

3. Step-by-Step Guide

Step 1: Create Directory

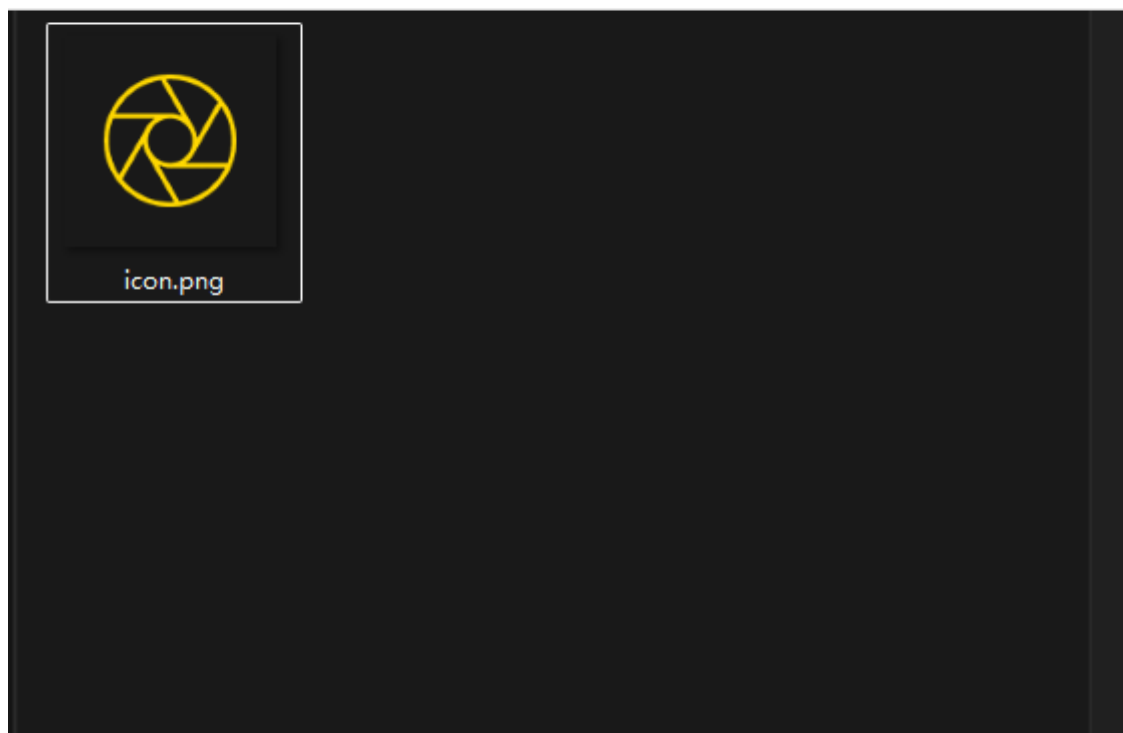
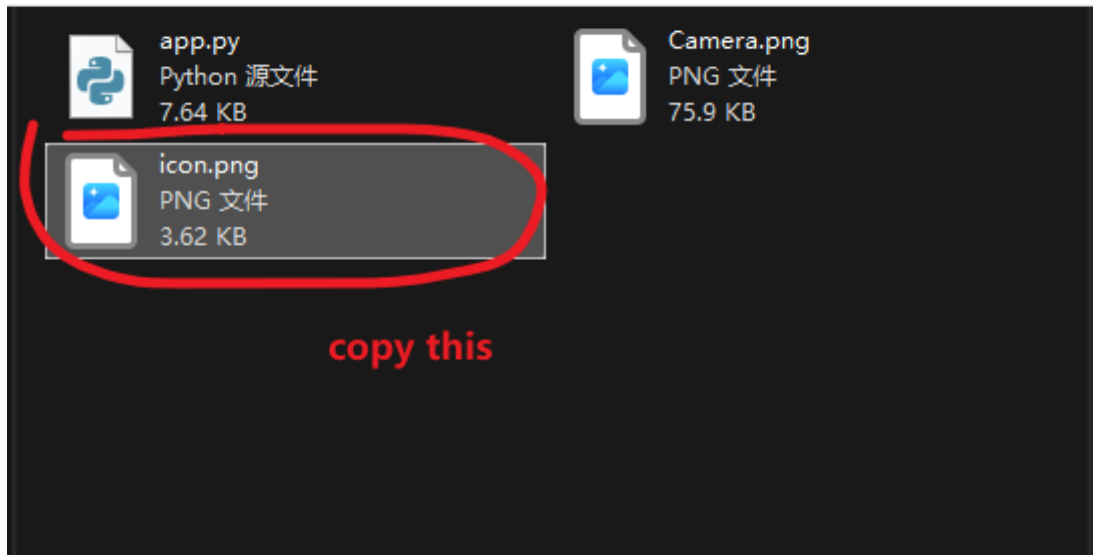
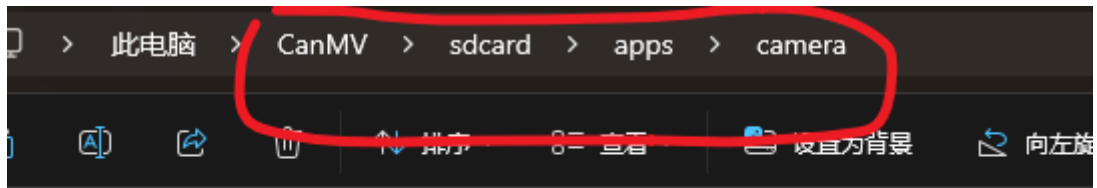
We cannot directly perform file operations under `/sdcard` (very inconvenient), so our subsequent coding work will be done in any folder on the computer. After completing the code, we will move the code files (folder) to the `/sdcard/apps/hello_world` directory.

Create a folder named `hello_world` in any folder on your computer.



Step 2: Prepare Icon

Find a PNG image (recommended 64x64 pixels), rename it to `icon.png`, and put it in the `hello_world` folder. If you don't have an image, you can temporarily copy the icon from the `camera` application.



Step 3: Write Code (app.py)

Create `app.py` in the `hello_world` folder and enter the following code:

```
import lvgl as lv
from yahboom.ybMain.base_app import BaseApp

class App(BaseApp):
    def __init__(self, app_manager):
        # Load image data
        try:
            with open("/sdcard/apps/hello_world/icon.png", 'rb') as f:
                bg_image_cache = f.read()
                img_bg = lv.img_dsc_t({
                    'data_size': len(bg_image_cache),
                    'data': bg_image_cache
                })
        except Exception as e:
            print(f"Failed to load icon image: {e}")
            img_bg = None

        # 1. Call parent class initialization
        # app_manager: Manager object passed in by the system
        # "Hello world": Name displayed below the desktop icon
        # icon: Icon path, here we simplify and not pass the image object,
        # BaseApp will handle the default case or you need to manually load the image
        super().__init__(app_manager, "Hello world", icon=img_bg)

    def initialize(self):
        # 2. When the application is opened, the system will automatically call
        # this method
        # self.screen is your canvas (full screen)

        # Create a label
        label = lv.label(self.screen)
        label.set_text("Hello CanMV K230!")

        # Set font (use system's built-in 16-point font)
        # Note: app_manager stores many global resources
        if hasattr(self.app_manager, "font_16"):
            label.set_style_text_font(self.app_manager.font_16, 0)

        # Center display
        label.center()

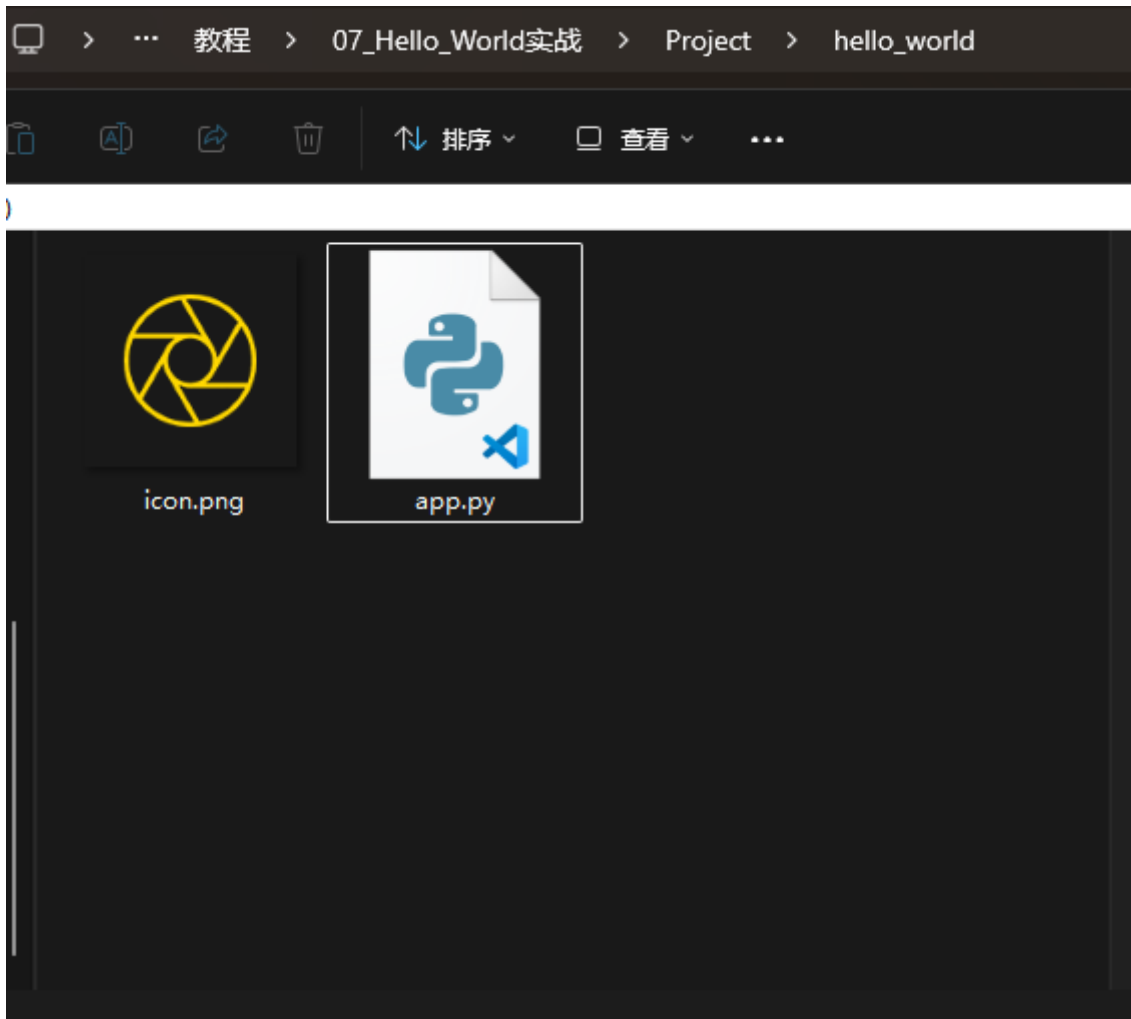
        # Create a button
        btn = lv.btn(self.screen)
        btn.align(lv.ALIGN.CENTER, 0, 50) # 50 pixels below center

        btn_label = lv.label(btn)
        btn_label.set_text("Click Me")

        # Add click event
        btn.add_event(self.on_btn_click, lv.EVENT.CLICKED, None)

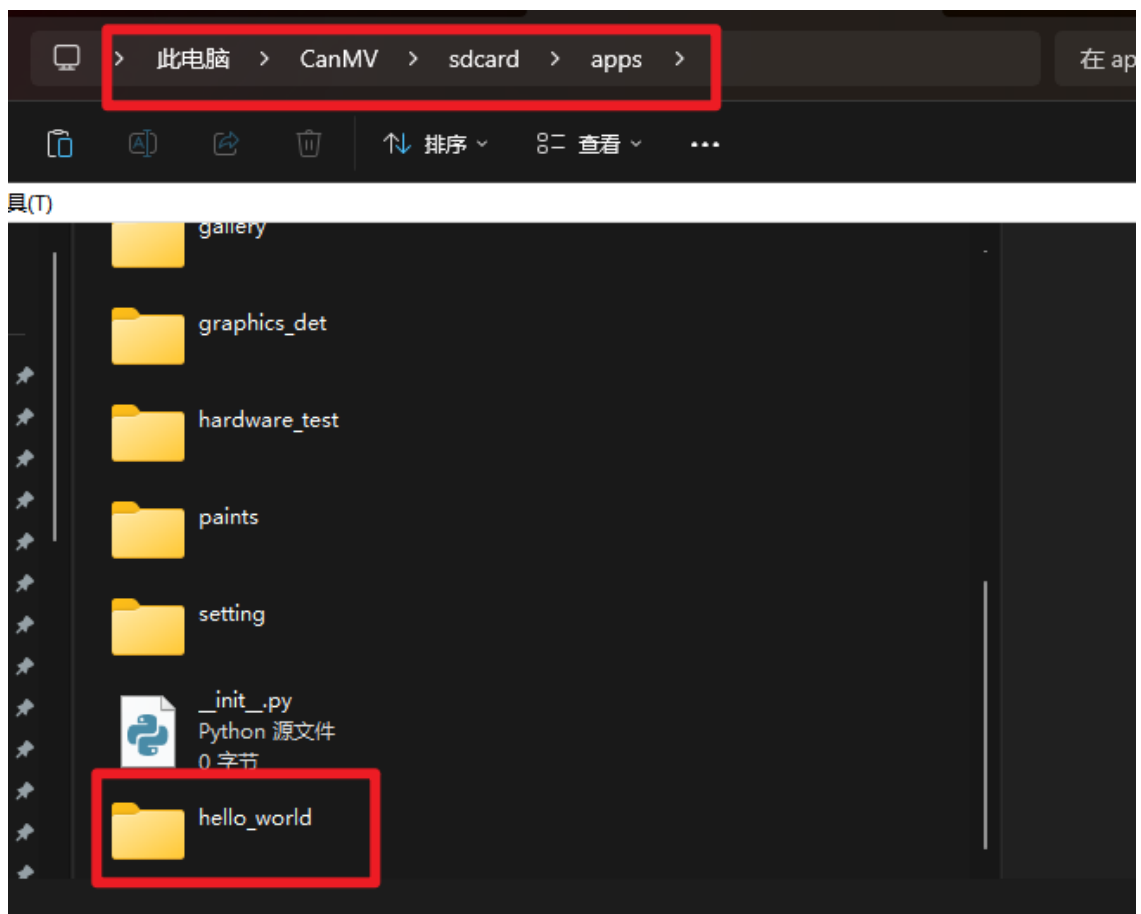
    def on_btn_click(self, event):
        print("Button Clicked!")
        # You can pop up a dialog here or modify the label's text
```

```
def on_close(self):  
    # Called when user clicks the return button in the top left corner  
    print("App Closed")  
    # Remember to clean up resources, although LVGL will automatically  
    recycle most objects
```

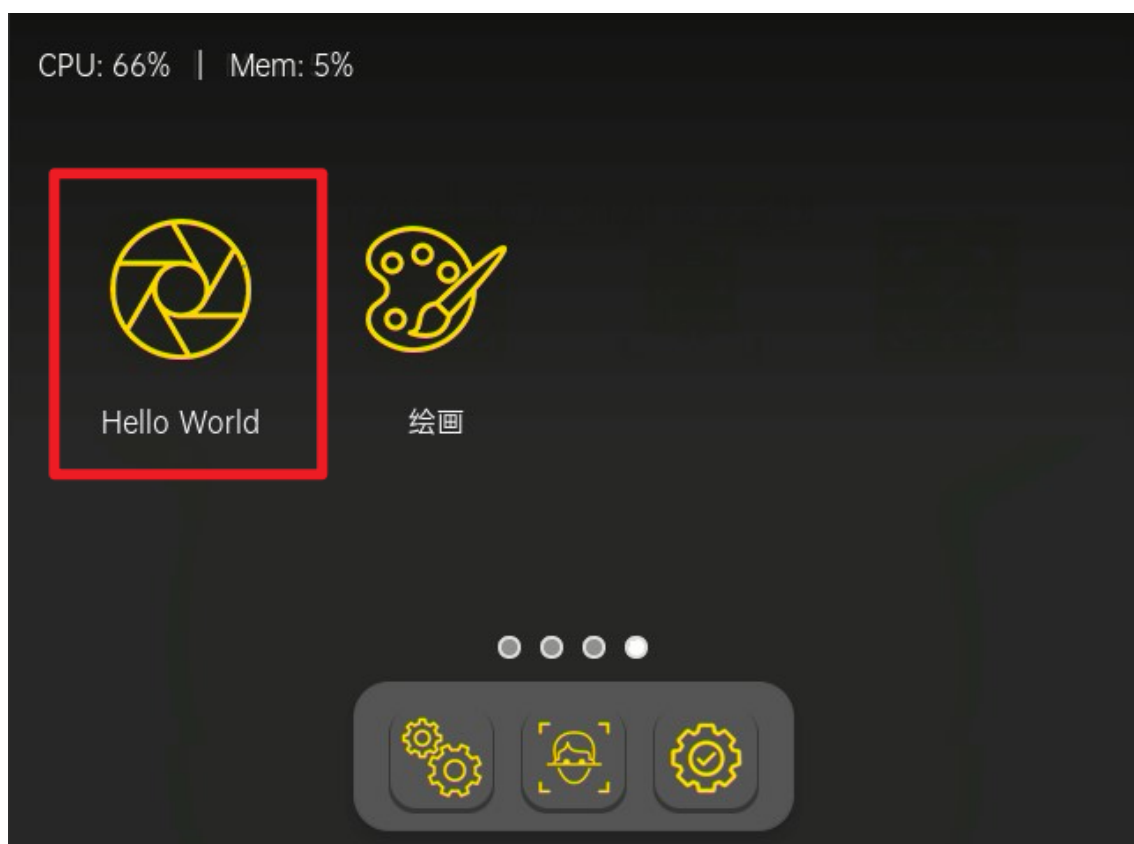


Step 4: Run Test

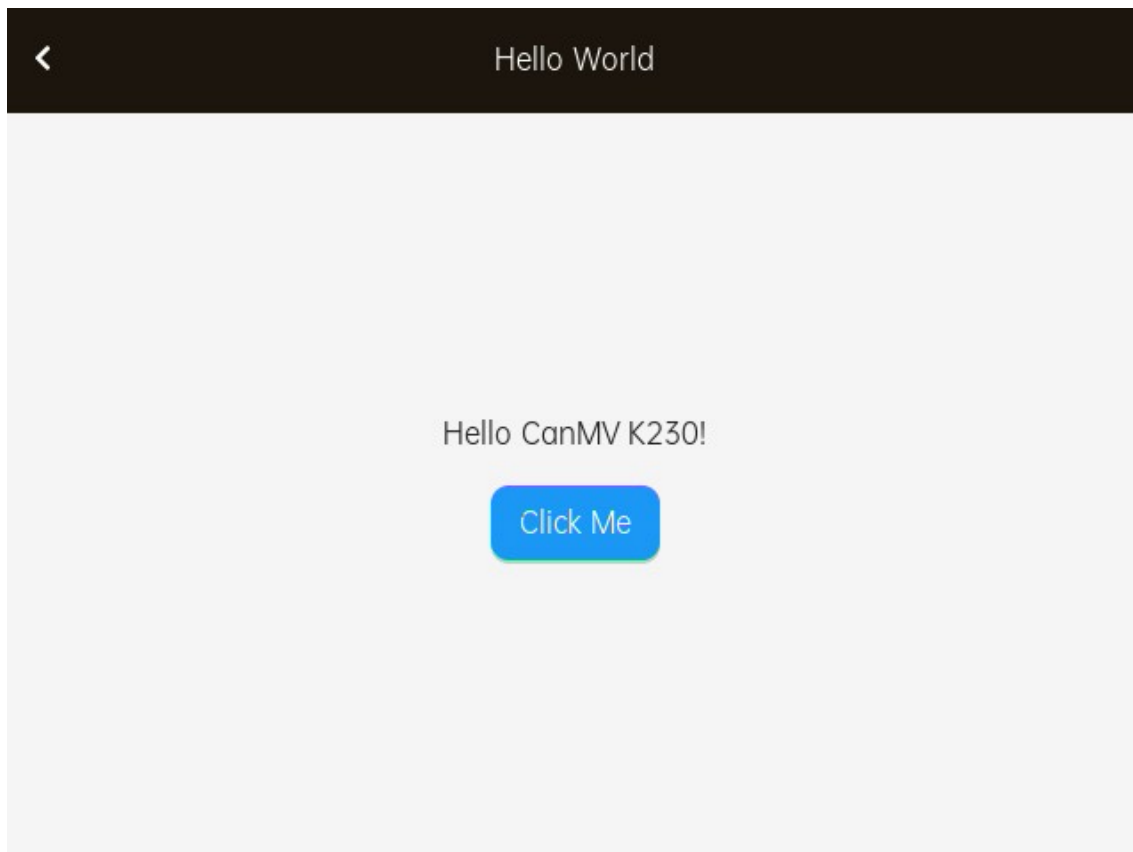
1. Save the file and move the hello_world folder from your computer to the /sdcard/apps directory



2. Restart the development board (or press the reset button).
3. Wait for the system to start, swipe the desktop, you should see a new "Hello World" icon.



4. Click the icon to verify if the interface displays correctly
If you see a screen like this, it means you wrote it successfully!



4. Common Troubleshooting

- **Icon not displayed?** Check if the folder name and `app.py` filename are correct.
- **Crashes on click?** Check if the `initialize` method name is spelled correctly.